

# Quoth Agent Memory System

Semantic Working Memory with Temporal Decay

Version 1.0 - March 2026

Architecture, API Reference, and Implementation Guide

# 1. Overview

---

The Quoth Agent Memory System provides semantic working memory for AI agents. Each agent has its own isolated memory space with vector embeddings for semantic search, temporal decay for relevance management, and automatic consolidation from working to persistent memory.

## Key Features:

- Per-agent isolated memory with namespace organization
- Semantic search via HNSW vector index (1536d, text-embedding-3-small)
- Two-tier memory: working (ephemeral) and persistent (consolidated)
- Temporal decay: unused memories fade, accessed ones strengthen
- Automatic consolidation via background worker (QStash or self-trigger)
- Full MCP tool integration for agent access
- REST API for programmatic access

# 2. Architecture

---

## 2.1 Database Schema

Agent memory is stored in the **agents.memory** table in Neon PostgreSQL with pgvector extension.

| Column           | Type               | Purpose                                        |
|------------------|--------------------|------------------------------------------------|
| id               | UUID               | Primary key (auto-generated)                   |
| agent_id         | UUID FK            | References agents.registry                     |
| org_id           | UUID FK            | Organization scope                             |
| project_id       | UUID FK (nullable) | Optional project scope                         |
| key              | TEXT               | Memory identifier (unique per agent+namespace) |
| value            | TEXT               | Memory content (string or serialized JSON)     |
| namespace        | TEXT               | Organization category (default: "default")     |
| embedding        | vector(1536)       | Semantic embedding via AI Gateway              |
| tier             | TEXT               | working or persistent                          |
| relevance_score  | FLOAT              | Current relevance (0.0 - 1.0)                  |
| access_count     | INTEGER            | Times accessed (search hits + direct gets)     |
| last_accessed_at | TIMESTAMPTZ        | Last access timestamp                          |

| Column     | Type                   | Purpose                                  |
|------------|------------------------|------------------------------------------|
| tags       | TEXT[]                 | Filterable tags array                    |
| metadata   | JSONB                  | Arbitrary metadata                       |
| source     | TEXT                   | Origin: agent, sync, consolidation, user |
| expires_at | TIMESTAMPTZ (nullable) | TTL expiration                           |

## 2.2 Indexes

| Index                           | Type   | Columns                               |
|---------------------------------|--------|---------------------------------------|
| idx_memory_embedding            | HNSW   | embedding (m=16, ef_construction=200) |
| idx_memory_agent_ns             | B-tree | agent_id, namespace                   |
| idx_memory_agent_tier           | B-tree | agent_id, tier                        |
| idx_memory_agent_tier_relevance | B-tree | agent_id, tier, relevance_score DESC  |
| idx_memory_relevance            | B-tree | agent_id, relevance_score DESC        |
| idx_memory_tags                 | GIN    | tags                                  |
| idx_memory_expires              | B-tree | expires_at (where not null)           |

## 2.3 Two-Tier Memory Model

**Working memory** is ephemeral. New memories start here with `relevance_score=1.0`. Every hour, working memories lose 5% relevance (multiplied by 0.95). If a working memory is accessed frequently (`access_count >= 3`) and maintains high relevance (`>= 0.5`), it gets promoted to **persistent memory** during consolidation. Working memories that fall below relevance 0.05 are automatically deleted.

Re-storing an existing key resets it to working tier, forcing it to re-earn persistence through use.

## 2.4 Relevance Scoring

Relevance is a dynamic score combining temporal decay with access frequency:

- **Initial store:** `relevance_score = 1.0`
- **Each access** (search hit or direct get): `+0.03` (capped at 1.0)
- **Hourly decay:** `relevance_score *= 0.95` (working tier only)
- **Search ranking:** `cosine_similarity * relevance_score` (combined score)
- **Cleanup threshold:** memories with `relevance < 0.05` are deleted
- **Consolidation threshold:** `access_count >= 3 AND relevance >= 0.5` promotes to persistent

## 3. API Reference

### 3.1 MCP Tools

| Tool                | Input                                                 | Description                                |
|---------------------|-------------------------------------------------------|--------------------------------------------|
| quoth_memory_store  | key, value, namespace?, tags?, metadata?, ttlSeconds? | Store or update a memory with embedding    |
| quoth_memory_search | query, namespace?, tier?, limit?, threshold?          | Semantic search across agent memories      |
| quoth_memory_list   | namespace?, tier?, limit?, offset?                    | List memories by namespace/tier            |
| quoth_memory_forget | key, namespace? OR namespace, all:true                | Delete specific memory or entire namespace |

### 3.2 REST API

| Method | Endpoint              | Auth  | Rate Limit | Description     |
|--------|-----------------------|-------|------------|-----------------|
| POST   | /api/v1/memory        | Agent | 60 rpm     | Store memory    |
| GET    | /api/v1/memory        | Agent | 60 rpm     | List memories   |
| POST   | /api/v1/memory/search | Agent | 120 rpm    | Semantic search |
| GET    | /api/v1/memory/{key}  | Agent | 60 rpm     | Get by key      |
| DELETE | /api/v1/memory/{key}  | Agent | 60 rpm     | Delete by key   |

## 4. Background Worker

The consolidation worker runs hourly (via QStash or self-trigger) and performs 5 tasks:

| Task               | SQL Function                    | Effect                                              |
|--------------------|---------------------------------|-----------------------------------------------------|
| 1. Temporal Decay  | agents.apply_memory_decay()     | Working memories * 0.95 relevance                   |
| 2. Consolidate     | agents consolidate_memory()     | Promote working -> persistent (access>=3, rel>=0.5) |
| 3. Cleanup         | agents.cleanup_memory()         | Delete expired + low relevance (<0.05)              |
| 4. Drift Detection | Custom query                    | Flag docs with stale memory references              |
| 5. Auto-fail Tasks | comms.auto_fail_overdue_tasks() | Fail tasks past deadline                            |

**Self-trigger (free tier):** maybeRunConsolidation() is called at the end of every search. It checks a Redis timestamp and runs consolidation if >1 hour since last run. Distributed lock prevents concurrent runs.

## 5. Embedding Pipeline

---

Every memory store generates a 1536-dimensional vector embedding via the Vercel AI Gateway using OpenAI's text-embedding-3-small model. This enables semantic search across agent memories.

- **Model:** openai/text-embedding-3-small (via Vercel AI Gateway)
- **Dimensions:** 1536
- **Index:** HNSW (m=16, ef\_construction=200, cosine distance)
- **Search:** agents.search\_memory() SQL function with threshold filtering
- **Cost:** \$0.02 per million tokens
- **Batch:** embedMany() supports up to 2048 inputs per call

## 6. Scoping and Isolation

---

| Scope        | Enforced By                      | Description                             |
|--------------|----------------------------------|-----------------------------------------|
| Agent        | agent_id FK + auth context       | Each agent has isolated memory space    |
| Organization | org_id FK                        | All queries scoped to authenticated org |
| Project      | project_id FK (optional)         | Optional project-level scoping          |
| Namespace    | UNIQUE(agent_id, namespace, key) | Logical grouping within agent memory    |